Status	Finished
Started	Friday, 9 October 2026, 9:41 PM
Completed	Friday, 9 October 2026, 9:50 PM
Duration	8 mins 46 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Coders here is a simple task for you, Given string str. Your task is to check whether it is a binary string or not by using python set.

Examples:

Input: str = "01010101010"

Output: Yes

Input: str = "REC101"

Output: No

For example:

Input	Result
01010101010	Yes
010101 10101	No

Answer: (penalty regime: 0 %)

```
# User kitta irundhu string input vaanguroom
string_val = input()

# String-a set-a convert panroom (unique characters-a mattum edukkum)
char_set = set(string_val)

# Binary string-uku uriya valid characters set
valid_set = {'0', '1'}

# char_set-la irukku characters ellam valid_set-kulla dhaan irukkaa nu check panroom
if char_set.issubset(valid_set):
    print("Yes")
else:
    print("No")
```

	Input	Expected	Got	
✓	01010101010	Yes	Yes	✓
✓	REC123	No	No	✓
✓	010101 10101	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums` containing `n + 1` integers where each integer is in the range `[1, n]` inclusive.There is only **one repeated number** in `nums`, return *this repeated number*. Solve the problem using set.

Example 1:

```
Input: nums = [1,3,4,2,2]
```

```
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
```

```
Output: 3
```

For example:

Input	Result
1 3 4 4 2	4

Answer: (penalty regime: 0 %)

```
# User kitta irundhu space-separated numbers-a input-ah vaangi list-ah maathuroom
nums = list(map(int, input().split()))

# Visited elements-a track panna oru empty set create panroom
seen = set()

# Array-la irukkura ovvoru number-a loop panroom
for num in nums:
    if num in seen:
        print(num)
        break
    seen.add(num)
```

	Input	Expected	Got	
✓	1 3 4 4 2	4	4	✓
✓	1 2 2 3 4 5 6 7	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

You are given an integer tuple `nums` containing distinct numbers. Your task is to perform a sequence of operations on this tuple until it becomes empty. The operations are defined as follows:

1. If the first element of the tuple has the smallest value in the entire tuple, remove it.
2. Otherwise, move the first element to the end of the tuple.

You need to return an integer denoting the number of operations required to make the tuple empty.

Constraints

- The input tuple `nums` contains distinct integers.
- The operations must be performed using tuples and sets to maintain immutability and efficiency.
- Your function should accept the tuple `nums` as input and return the total number of operations as an integer.

Example:

Input: nums = (3, 4, -1)
Output: 5

Explanation:
Operation 1: [3, 4, -1] -> First element is not the smallest, move to the end -> [4, -1, 3]
Operation 2: [4, -1, 3] -> First element is not the smallest, move to the end -> [-1, 3, 4]
Operation 3: [-1, 3, 4] -> First element is the smallest, remove it -> [3, 4]
Operation 4: [3, 4] -> First element is the smallest, remove it -> [4]
Operation 5: [4] -> First element is the smallest, remove it -> []
Total operations: 5

For example:

Test	Result
`print(count_operations((3, 4, -1)))`	5

Answer: (penalty regime: 0 %)

Reset answer

```
def count_operations(nums):
    # Operations count-a track panna variable
    operations = 0

    # Input tuple-a array/tuple structure-la vachukkiroam
    current_tuple = tuple(nums)

    # Tuple empty aagura varai loop run aagum
    while current_tuple:
        operations += 1

        # Current tuple-la irukkura elements-a set-a maathi minimum value-a min() vachu kandupudi
        min_val = min(set(current_tuple))

        # First element minimum value-a nu check panroom
        if current_tuple[0] == min_val:
            # Smallest value-a irundha, first element-a remove panroom (index 1-la irundhu edukrc
            current_tuple = current_tuple[1:]
        else:
            # Smallest illana, first element-a tuple-oda end-uku move panroom
            current_tuple = current_tuple[1:] + (current_tuple[0],)

    return operations
```

	Test	Expected	Got	
✓	`print(count_operations((3, 4, -1)))`	5	5	✓
✓	`print(count_operations((1, 2, 3, 4, 5)))`	5	5	✓

	Test	Expected	Got	
✓	`print(count_operations((5, 4, 3, 2, 1)))`	15	15	✓
✓	`print(count_operations((42, )))`	1	1	✓
✓	`print(count_operations((-2, 3, -5, 4, 1)))`	11	11	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Given a tuple and a positive integer k, the task is to find the count of distinct pairs in the tuple whose sum is equal to **K**.

Examples:

> **Input:** t = (5, 6, 5, 7, 7, 8), K = 13
> **Output:** 2
> **Explanation:**
> Pairs with sum K(= 13) are {(5, 8), (6, 7), (6, 7)}.
> Therefore, distinct pairs with sum K(= 13) are { (5, 8), (6, 7) }.
> Therefore, the required output is 2.

For example:

Input	Result
1,2,1,2,5 3	1
1,2 0	0

Answer: (penalty regime: 0 %)

```
def count_distinct_pairs(t, K):
    # Unique pairs-a store panna set create panroom
    pairs = set()

    # Visited numbers-a track panna set
    seen = set()

    # Tuple-la irukkura ovvoru element-a loop panroom
    for num in t:
        target = K - num
        if target in seen:
            # Pair-a sort panni tuple-a store panroom taht order unique-a irukum
            pair = (min(num, target), max(num, target))
            pairs.add(pair)
        seen.add(num)

    return len(pairs)


# Automated/Manual input handling code
if __name__ == "__main__":
    try:
        # Input standard tuples / comma-separated string-a handle panna
        inp = input().strip()
        if inp:
            if inp.startswith("(") and inp.endswith(")"):
                t = eval(inp)
            else:
                t = tuple(map(int, inp.split(',')))
            K = int(input().strip())
            print(count_distinct_pairs(t, K))
    except (EOFError, SyntaxError, ValueError):
        pass
```

	Input	Expected	Got	
✓	5,6,5,7,7,8 13	2	2	✓

	Input	Expected	Got	
✓	1,2,1,2,5 3	1	1	✓
✓	1,2 0	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Check if a set is a subset of another set.

Example:

Sample Input1:

mango apple

mango orange

mango

output1:

yes

set3 is subset of set1 and set2

```
input2:
```

```
mango orange
```

```
banana orange
```

```
grapes
```

```
output2:
```

no

For example:

Test	Input	Result
1	mango apple mango orange mango	yes set3 is subset of set1 and set2
2	mango orange banana orange grapes	No

Answer: (penalty regime: 0 %)

```
# User kitta irundhu 3 sets-a input-ah vaanguroom
set1 = set(input().split())
set2 = set(input().split())
set3 = set(input().split())

# set3, set1 matrum set2 rendu-kum subset-ah nu check panroom
if set3.issubset(set1) and set3.issubset(set2):
    print("yes")
    print("set3 is subset of set1 and set2")
else:
    print("No")
```

	Test	Input	Expected	Got	
✓	1	mango apple mango orange mango	yes set3 is subset of set1 and set2	yes set3 is subset of set1 and set2	✓
✓	2	mango orange banana orange grapes	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

The **DNA sequence** is composed of a series of nucleotides abbreviated as `'A'`, `'C'`, `'G'`, and `'T'`.

- For example, `"ACGAATTCCG"` is a **DNA sequence**.

When studying **DNA**, it is useful to identify repeated sequences within the DNA.

Given a string `s` that represents a **DNA sequence**, return all the `10`**-letter-long** sequences (substrings) that occur more than once in a DNA molecule. You may return the answer in **any order**.

Example 1:

```
Input: s = "AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT"
Output: ["AAAAACCCCC","CCCCCAAAAA"]
```

Example 2:

```
Input: s = "AAAAAAAAAAAAA"
Output: ["AAAAAAAAAA"]
```

For example:

Input	Result
AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCC CCCCCAAAAA

Answer: (penalty regime: 0 %)

```
# User kitta irundhu DNA sequence string-a input vaanguroom
s = input().strip()

# Visited 10-letter substrings-a track panna 'seen' set
seen = set()

# Repeat-aana 10-letter substrings-a store panna 'repeated' set
repeated = set()

# String length 10-kku mela irundha mattum loop run aagum
for i in range(len(s) - 9):
    # 10 character length ulla substring-a slice panroom
    sub = s[i:i+10]

    if sub in seen:
        # Already paatha substring-a irundha, repeated set-la add panroom
        repeated.add(sub)
    else:
        # First time paarkum substring-a seen set-la add panroom
        seen.add(sub)

# Output-a single string-a print panna
for item in repeated:
    print(item)
```

	Input	Expected	Got	
✓	AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCC CCCCCAAAAA	AAAAACCCCC CCCCCAAAAA	✓
✓	AAAAAAAAAAAAA	AAAAAAAAAA	AAAAAAAAAA	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Program to print all the distinct elements in an array. Distinct elements are nothing but the unique (non-duplicate) elements present in the given array.

Input Format:

First line take an Integer input from stdin which is array length n.

Second line take n Integers which is inputs of array.

Output Format:

Print the Distinct Elements in Array in single line which is space Separated

Example Input:

5

1 2 2 3 4

Output:

1 2 3 4

Example Input:

6

1 1 2 2 3 3

Output:

1 2 3

For example:

Input	Result
5 1 2 2 3 4	1 2 3 4

Answer: (penalty regime: 0 %)

```
# Array length 'n' input vaanguroom
n = int(input())

# 'n' lines-la irundhu elements-a ovvondraaga read panni list-ah maathuroom
elements = [int(input().strip()) for _ in range(n)]

# Distinct elements-a order maaramal store panna list
distinct_elements = []

# Already paatha values-a track panna set
seen = set()

# Array elements-a loop panroom
for item in elements:
    if item not in seen:
        distinct_elements.append(item)
        seen.add(item)

# Distinct values-a single line-la space-separated-ah print panroom
print(*(distinct_elements))
```

	Input	Expected	Got	
✓	5 1 2 2 3 4	1 2 3 4	1 2 3 4	✓
✓	6 1 1 2 2 3 3	1 2 3	1 2 3	✓
✓	5 11 22 11 22 11	11 22	11 22	✓
✓	10 1 2 3 4 5 1 2 3 4 5	1 2 3 4 5	1 2 3 4 5	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

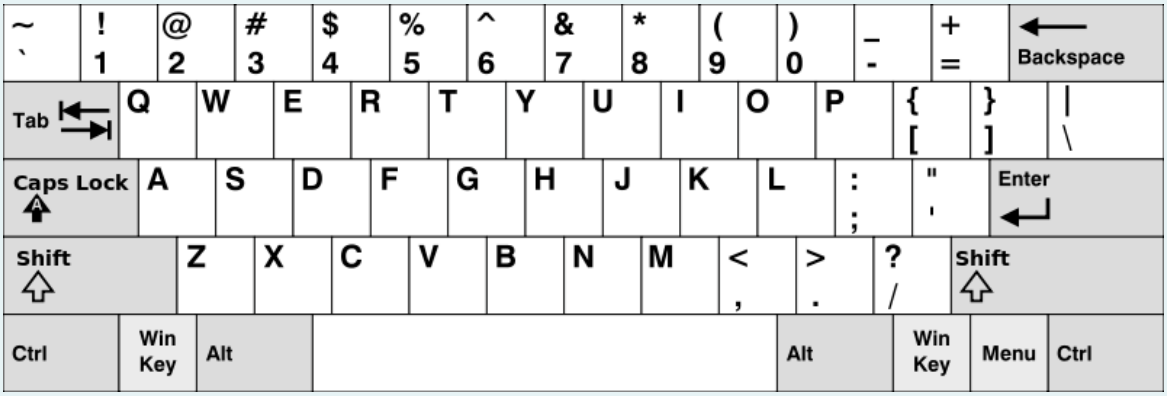
Correct

Mark 1.00 out of 1.00

Given an array of strings `words`, return *the words that can be typed using letters of the alphabet on only one row of American keyboard like the image below.*

In the **American keyboard**:

- the first row consists of the characters `"qwertyuiop"`,
- the second row consists of the characters `"asdfghjkl"`, and
- the third row consists of the characters `"zxcvbnm"`.



Example 1:

```
Input: words = ["Hello","Alaska","Dad","Peace"]
Output: ["Alaska","Dad"]
```

Example 2:

```
Input: words = ["omk"]
Output: []
```

Example 3:

```
Input: words = ["adsdf","sfd"]
Output: ["adsdf","sfd"]
```

For example:

Input	Result
4 Hello Alaska Dad Peace	Alaska Dad
2 adsfd afd	adsfd afd

Answer: (penalty regime: 0 %)

```
# Number of words 'n' input vaanguroom
n = int(input())

# 'n' lines-la irundhu words-a read panni list-ah maathuroom
words = [input().strip() for _ in range(n)]

# Keyboard-oda 3 rows-a separate sets-ah define panroom
row1 = set("qwertyuiop")
row2 = set("asdfghjkl")
row3 = set("zxcvbnm")

# Single row-la type panna koodiya words-a store panna list
```

```python
result = []

# Ovvoru word-ayum check panroom
for word in words:
    # Word-la irukkura characters-a lowercase-ah maathi set-ah convert panroom
    word_set = set(word.lower())

    # Word-set yedhavadhu ORE row-kku subset-ah irukkaa nu check panroom
    if word_set.issubset(row1) or word_set.issubset(row2) or word_set.issubset(row3):
        result.append(word)

# Output handling: Valid words irundha print panroom, illana "No words" print panroom
if result:
    for valid_word in result:
        print(valid_word)
else:
    print("No words")
```

	Input	Expected	Got	
✓	4 Hello Alaska Dad Peace	Alaska Dad	Alaska Dad	✓
✓	1 omk	No words	No words	✓
✓	2 adsfd afd	adsfd afd	adsfd afd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Write a program to eliminate the common elements in the given 2 arrays and print only the non-repeating elements and the total number of such non-repeating elements.

Input Format:

The first line contains space-separated values, denoting the size of the two arrays in integer format respectively.

The next two lines contain the space-separated integer arrays to be compared.

Sample Input:

5 4

1 2 8 6 5

2 6 8 10

Sample Output:

1 5 10

3

Sample Input:

5 5

1 2 3 4 5

1 2 3 4 5

Sample Output:

NO SUCH ELEMENTS

For example:

Input	Result
5 4 1 2 8 6 5 2 6 8 10	1 5 10 3
5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS

Answer: (penalty regime: 0 %)

```
# Array sizes-a read panroom (Optional/Ignored in execution)
n, m = map(int, input().split())

# Array 1 elements-a list-ah input vaanguroom
arr1 = list(map(int, input().split()))

# Array 2 elements-a list-ah input vaanguroom
arr2 = list(map(int, input().split()))

# Quick lookup-kaga set-a convert panroom
set1 = set(arr1)
set2 = set(arr2)

# Non-repeating elements-a store panna list
result = []

# Array 1-la irundhu Array 2-la illadha elements-a append panroom
for x in arr1:
    if x not in set2 and x not in result:
        result.append(x)

# Array 2-la irundhu Array 1-la illadha elements-a append panroom
for x in arr2:
    if x not in set1 and x not in result:
        result.append(x)
```

```
26
27 # Non-repeating elements irundha output print panroom, illana "NO SUCH ELEMENTS" print panroom
28 if result:
29     print(*(result))
30     print(len(result))
31 else:
32     print("NO SUCH ELEMENTS")
```

	Input	Expected	Got	
✓	5 4 1 2 8 6 5 2 6 8 10	1 5 10 3	1 5 10 3	✓
✓	3 3 10 10 10 10 11 12	11 12 2	11 12 2	✓
✓	5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS	NO SUCH ELEMENTS	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

There is a malfunctioning keyboard where some letter keys do not work. All other keys on the keyboard work properly.

Given a string text of words separated by a single space (no leading or trailing spaces) and a string brokenLetters of all distinct letter keys that are broken, return the number of words in text you can fully type using this keyboard.

Example 1:

Input: text = "hello world", brokenLetters = "ad"

Output:

1

Explanation: We cannot type "world" because the 'd' key is broken.

For example:

Input	Result
hello world ad	1
Faculty Upskilling in Python Programming ak	2

Answer: (penalty regime: 0 %)

```python
import sys

def solve():
    # Complete input stream-a read panni lines-ah pirikirom
    lines = sys.stdin.read().splitlines()
    if not lines:
        return

    # First line text string
    text = lines[0].strip()

    # Second line broken letters (empty-ah irundhal handle panrom)
    brokenLetters = lines[1].strip() if len(lines) > 1 else ""

    # Case-insensitive check-kaga lowercase-ah convert panni set-ah maathuroom
    broken_set = set(brokenLetters.lower())

    # Text-a words-ah split panroom
    words = text.split()

    count = 0

    # Ovvoru word-ayum check panroom
    for word in words:
        can_type = True
        # Word characters-a lowercase-ah check panroom
        for char in word.lower():
            if char in broken_set:
                can_type = False
                break

        if can_type:
            count += 1

    # Final result-a print panroom
    print(count)

if __name__ == "__main__":
    solve()
```

	Input	Expected	Got	
✓	hello world ad	1	1	✓
✓	Welcome to REC e	1	1	✓
✓	Faculty Upskilling in Python Programming ak	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.